

Level 2.6 - Detect ARP poisoning (Dynamic definition)

Now we will upgrade the NIDS to detect all IP:MAC mappings in runtime by learning the normal network traffic.

The NIDS will start a 30-seconds learning phase upon startup, during which it will count IP:MAC pairing instances in ARP response packets. When this period ends, it will decide what MAC is expected per IP, simply by selecting the most common MAC for this IP (when there is no ARP poisoning in effect, there should only be one MAC. But if there is an ARP poisoning attempt during the learning period, it will still probably be less common than the real server's MAC)

Password

gh39idcns1

Instructions

1. Add a new module `learning.py` with a class `DynamicLearner` and two methods
 - This class will be **generic** - no mention of ARP; it is simply for learning key:value pairs
 - `start_learning(self, learning_time)`: Define internally the end time for the learning phase
 - `is_expected(self, key, value)`: If we are still in the learning phase, just count this key:value pair and return True. If we are past the learning phase, return True if the value is expected for this key, and False otherwise
2. In the main module, create an object of the class and call `start_learning` on the NIDS startup. Then, for each ARP response sniffed, pass the contained answer's IP and MAC to the `DynamicLearner` object, and alert if it returns False.

3. Remember to disable the static definition detection (level 2.5) to make sure this new technique works!

4. To trigger the attack

- Make sure there is no ARP poisoning attack ongoing on the attacker (if there is, click "Stop")
- Run the NIDS and wait 30 seconds. The client resolves the server's MAC every 10 seconds, so during the learning phase, the NIDS learns the server's correct MAC based on the responses.
- After the learning phase finishes, run the attacker's malicious trigger for level 2.6 - this will start the ARP poisoning (You may stop it at any point with the "Stop" button)
- Then, run the client's non-malicious trigger for level 2.6 - it should be poisoned by the attacker

Notes

Use the guiding questions below to identify the malicious trigger and structure your code's detection:

- How do we check if the program is still on its learning phase or not?
- What data structure should we use to keep track of IP and MAC address mappings?

To submit

Submit a ZIP `nids.zip` containing all `.py` files.

